

Wireshark Assignment

NAME: Zhuo Han

EID: zhuohan3

SID: 60130387

EMAIL: zhuo.han@my.cityu.edu.hk

November 28, 2025

Problem 1: Capturing Web Browser HTTP Traffic

I follow steps 1 to 6 and have captured two HTTP packets, as shown in Figure 1.

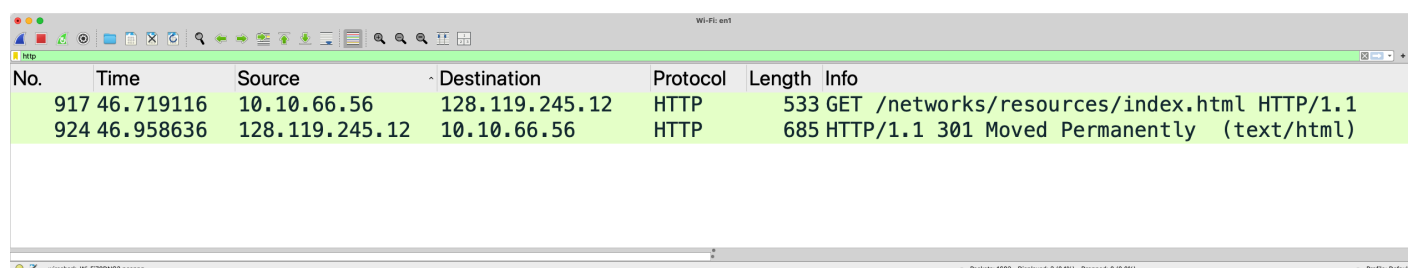


Figure 1: The two packets captured after accessing <http://gaia.cs.umass.edu/networks/resources/index.html>.

Q1: The HTTP GET request indicates the browser can accept Simplified Chinese (China) and Chinese (generic) via the header, which is shown in Figure 2a. As illustrated, the light blue background line shows **Accept-Language: zh-CN,zh;q=0.9**.

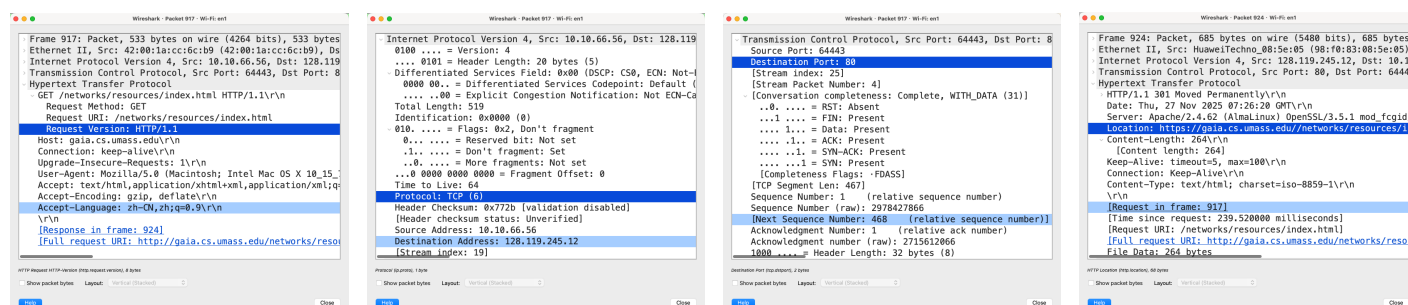
Q2: The application layer protocol used to access the website is HTTP/1.1, as shown in the request line illustrated in Figure 2a. As illustrated, the dark blue background line shows **Request Version: HTTP/1.1**.

Q3: The transport layer protocol used by this HTTP GET packet is TCP, as shown in Figure 2b. As illustrated, the dark blue background line shows **Protocol: TCP (6)**.

Q4: The destination IP address and port of the HTTP GET message are 128.119.245.12:80, as shown in Figure 2b and Figure 2c. As illustrated, the light blue background line in Figure 2b and the dark blue background line in Figure 2c show the destination address is **Destination Address: 128.119.245.12** and the port is **Destination Port: 80**, respectively.

Q5: The next sequence number in the TCP header is 468, as shown in Figure 2c. As illustrated, the light blue background line in Figure 2c shows **[Next Sequence Number: 468 (relative sequence number)]**.

Q6: The response packet corresponding to the HTTP GET message is packet 924: **HTTP/1.1 301 Moved Permanently (text/html)** in Figure 1. As shown in Figure 2d, the light blue background line shows explicitly this answers the request in frame 917, and the dark blue background line shows the redirected target.



(a) The Hypertext Transfer Protocol field of packet 917. (b) The Internet Protocol field of packet 917. (c) The Transmission Control Protocol field of packet 917. (d) The Hypertext Transfer Protocol field of packet 924.

Figure 2: Details of packet 917 and 924. The highlighted lines are the key fields based on which the questions are answered.

The file `problem_1.pcapng` has two rounds of HTTP interactions, as shown in Figure 3.

Q7: The first HTTP GET packet does not include the **IF-MODIFIED-SINCE** line, as shown in Figure 4a.

Q8: Yes, the server explicitly returned the file contents. As shown in Figure 4b, the response is **HTTP/1.1 200 OK** and the entity body shows **Content-Length: 20146** with **Content-Type: application/pdf**.

No.	Time	Source	Protocol	Length	Destination	Info
247	2.918999	192.168.3.4	HTTP	523	128.119.245.12	GET /networks/cnrg_wireless-10-02.pdf HTTP/1.1
298	3.390406	128.119.245.12	HTTP	965	192.168.3.4	HTTP/1.1 200 OK (application/pdf)
309	5.772887	192.168.3.4	HTTP	636	128.119.245.12	GET /networks/cnrg_wireless-10-02.pdf HTTP/1.1
310	6.042193	128.119.245.12	HTTP	306	192.168.3.4	HTTP/1.1 304 Not Modified

Figure 3: The four packets of two rounds of HTTP interactions in problem_1.pcapng.

- Q9:** Yes, there is a IF-MODIFIED-SINCE line in the second HTTP GET request. And as Figure 4c shows, the specific content is IF-MODIFIED-SINCE: Tue, 27 Jan 2004 21:48:52 GMT.
- Q10:** The status is 304 Not Modified. And the server does not explicitly return the contents of the file, as shown in Figure 4d, there are no lines representing content type and content length in the response message.

Frame 247: Packet, 523 bytes on wire (4184 bits), 523 bytes captured (4184 bits) on interface 0 Ethernet II, Src: Apple78:ab:af (ac:bc:32:78:ab:af), Dst: 128.119.245.12 Internet Protocol Version 4, Src: 192.168.3.4, Dst: 128.119.245.12 Transmission Control Protocol, Src Port: 62900, Dst Port: 80 Hypertext Transfer Protocol GET /networks/cnrg_wireless-10-02.pdf HTTP/1.1\r\n Host: gaia.cs.umass.edu\r\n Connection: keep-alive\r\n Upgrade-Insecure-Requests: 1\r\n User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_7_0; rv:3.0) Gecko/20100101 Firefox/3.0\r\n Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n Accept-Encoding: gzip, deflate\r\n Accept-Language: zh-TW,zh;q=0.9,en-US;q=0.8,en;q=0.7\r\n [Response in frame: 298] [Full request URI: http://gaia.cs.umass.edu/networks/cnrg_wireless-10-02.pdf]	Frame 298: Packet, 965 bytes on wire (7720 bits), 965 bytes captured (7720 bits) on interface 0 Ethernet II, Src: HuaweiTechno_c2:0f:89 (7c:ai:77:c2:0f:89), Dst: 192.168.3.4 Internet Protocol Version 4, Src: 128.119.245.12, Dst: 192.168.3.4 Transmission Control Protocol, Src Port: 80, Dst Port: 62900 Hypertext Transfer Protocol HTTP/1.1 200 OK\r\n Date: Fri, 19 Oct 2018 02:51:41 GMT\r\n Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips\r\n Last-Modified: Tue, 27 Jan 2004 21:48:52 GMT\r\n ETag: "4eb2-3d1ecd1db900"\r\n Accept-Ranges: bytes\r\n Content-Length: 20146\r\n [Content length: 20146] Keep-Alive: timeout=5, max=100\r\n Connection: Keep-Alive\r\n Content-Type: application/pdf\r\n [Full response URI: http://gaia.cs.umass.edu/networks/cnrg_wireless-10-02.pdf]	Frame 309: Packet, 636 bytes on wire (5088 bits), 636 bytes captured (5088 bits) on interface 0 Ethernet II, Src: Apple78:ab:af (ac:bc:32:78:ab:af), Dst: 128.119.245.12 Internet Protocol Version 4, Src: 192.168.3.4, Dst: 128.119.245.12 Transmission Control Protocol, Src Port: 62900, Dst Port: 80 Hypertext Transfer Protocol GET /networks/cnrg_wireless-10-02.pdf HTTP/1.1\r\n Host: gaia.cs.umass.edu\r\n Connection: keep-alive\r\n Upgrade-Insecure-Requests: 1\r\n User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_7_0; rv:3.0) Gecko/20100101 Firefox/3.0\r\n Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n Accept-Encoding: gzip, deflate\r\n Accept-Language: zh-TW,zh;q=0.9,en-US;q=0.8,en;q=0.7\r\n If-Modified-Since: Tue, 27 Jan 2004 21:48:52 GMT\r\n [Response in frame: 310] [Full request URI: http://gaia.cs.umass.edu/networks/cnrg_wireless-10-02.pdf]	Frame 310: Packet, 306 bytes on wire (2448 bits), 306 bytes captured (2448 bits) on interface 0 Ethernet II, Src: HuaweiTechno_c2:0f:89 (7c:ai:77:c2:0f:89), Dst: 192.168.3.4 Internet Protocol Version 4, Src: 128.119.245.12, Dst: 192.168.3.4 Transmission Control Protocol, Src Port: 80, Dst Port: 62900 Hypertext Transfer Protocol HTTP/1.1 304 Not Modified\r\n Date: Fri, 19 Oct 2018 02:51:44 GMT\r\n Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips\r\n Connection: Keep-Alive\r\n Keep-Alive: timeout=5, max=99\r\n ETag: "4eb2-3d1ecd1db900"\r\n [Request in frame: 309] [Time since request: 269.306000 milliseconds] [Request URI: /networks/cnrg_wireless-10-02.pdf] [Full request URI: http://gaia.cs.umass.edu/networks/cnrg_wireless-10-02.pdf]
---	---	---	--

- (a) The Hypertext Transfer Protocol field of the first GET packet.
- (b) The Hypertext Transfer Protocol field of the first response packet.
- (c) The Hypertext Transfer Protocol field of the second GET packet.
- (d) The Hypertext Transfer Protocol field of the second response packet.

Figure 4: Details of the four packets provided in problem_1.pcapng.

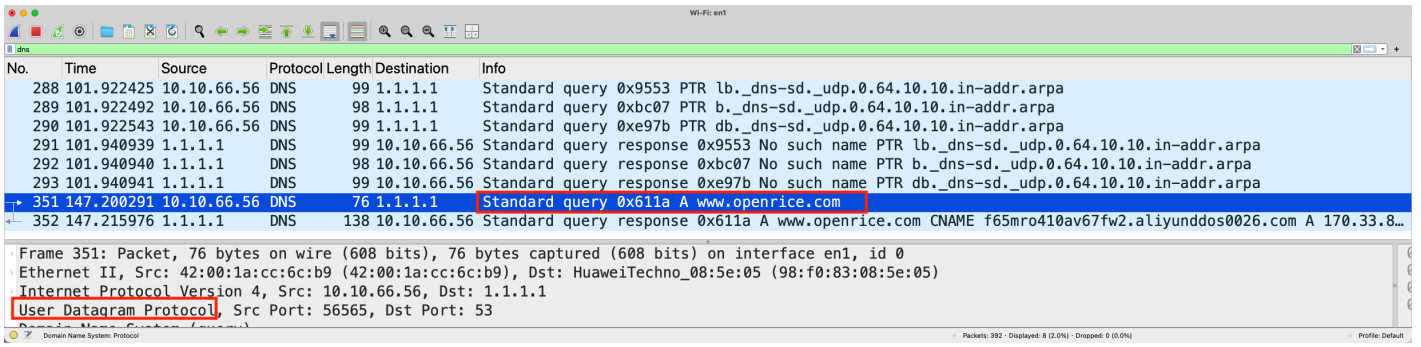


Figure 5: The DNS packets captured after looking up www.openrice.com.

Problem 2: Capturing DNS Traffic

I follow the steps 1 to 6 and have captured several DNS packets, including the query and response of www.openrice.com. Moreover, as I am operating on MacOS, the first two command I used in my terminal is

```
sudo dscacheutil -flushcache
```

(1)

```
sudo killall -HUP mDNSResponder
```

(2)

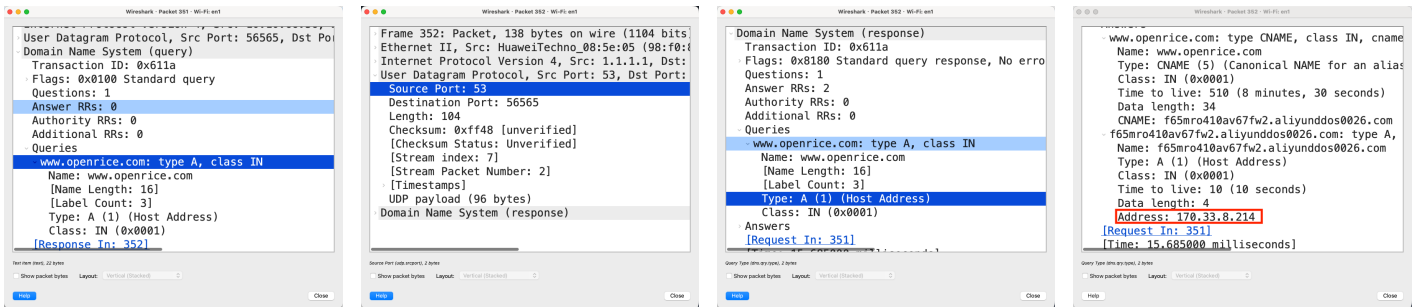
Q1: They are sent over UDP. As shown in Figure 5 and highlighted in red, the query message includes the UDP field.

Q2: The DNS field of the message body of this packet is shown in Figure 6a. As shown in the figure, the DNS query is a type A. And the query message does not contain the answer because the message has Answer RRs: 0.

Q3: The source port is 53 (see Figure 6b), and the type of address is type A (see Figure 6c).

Q4: The IP address is 170.33.8.214. This is presented within the response packet, as shown in Figure 6d.

Q5: As shown in Figure 7a, the destination IP address of the DNS query is 1.1.1.1, which is the same as my local DNS (see Figure 7b).



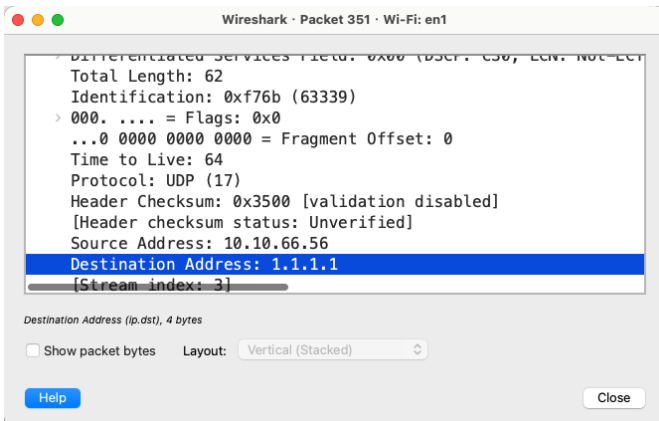
(a) The DNS field of the query packet (packet 351).

(b) The User Datagram Protocol field of the response packet (packet 352).

(c) The detailed information inside the DNS field of packet 352, showing the type of the address.

(d) The detailed information inside the DNS field of packet 352, showing the actual IP address of www.openrice.com.

Figure 6: Details of the packages captured during the DNS query.



(a) The destination IP address of DNS.



(b) My local DNS information.

Figure 7: Information of the DNS servers.

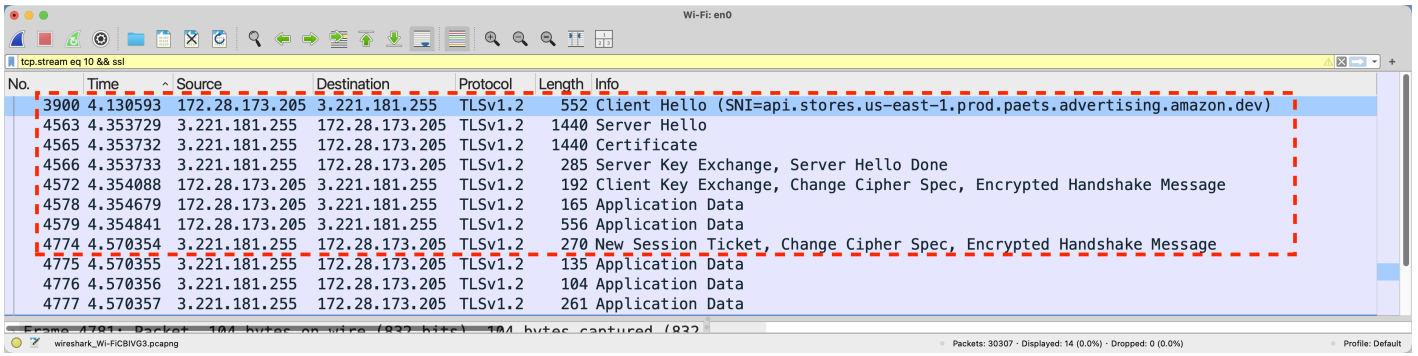


Figure 8: SSL packets captured.

Problem 3: Capturing SSL Traffic

After halting purchase before payment at **amazon.com**, I've captures a series of SSL packets as shown in Figure 8.

Q1: The first 8 frames of this session is shown in Table 1. Specifically, the SSL count of each frame is determined by the number of TLS record layers inside the TLS field of the message. The IP address of the client is 172.28.173.205, and that of the server is 3.221.181.255. In the first handshake packet, the frame labeled **Client Hello** is sent from 172.28.173.205 to 3.221.181.255, as shown in Figure 8. The host that initiates the handshake is the client, and the host replying with **Server Hello/Certificate** is the server.

Q2: The three fields and their lengths are: **Content Type** - 1 Byte, **Version** - 2 Bytes, and **Length** - 2 Bytes.

Q3: The value of the content type is **Handshake** (22) (see Figure 9a). It tells the receiver that this record contains a handshake message, so the receiver parses the record payload as handshake protocol data.

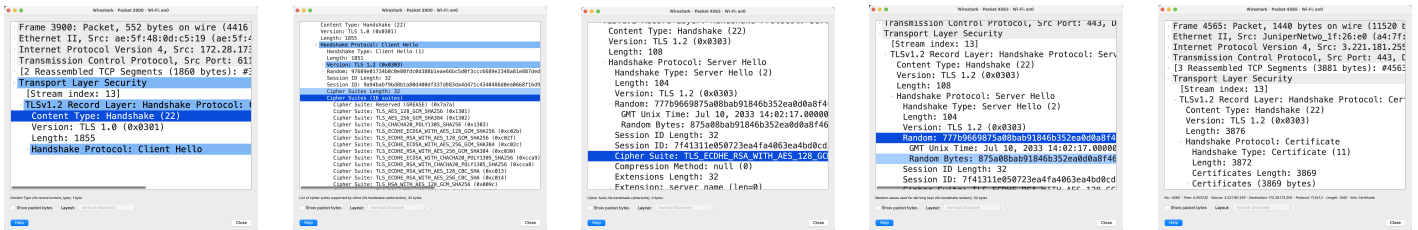
Q4: Yes, the Client Hello record advertises the cipher suites it supports. Details are shown in Figure 9b.

Q5: Yes, the cipher chosen is shown in Figure 9c.

Q6: The Server Hello includes a nonce in the Random field (Figure 9d), which is 32 bytes long. The client and server nonces ensure handshake freshness and contribute to session key derivation, preventing key reuse and replay attacks.

Q7: Yes, as Figure 9d shows **Session ID Length: 32**, followed by the **Session ID** value. The session ID identifies the session and is used for session resumption.

Q8: No. The certificate is carried in a separate handshake message, as seen in Figure 8. And no the certificate here is fragmented across 3 TCP segments, as Figure 9e shows.



(a) Details of Client Hello message.

(b) Details of Client Hello message, showing the cipher suites.

(c) Details of Server Hello message, showing the cipher suite.

(d) Details of Server Hello message, showing the nonce.

(e) Details of the Certificate message.

Figure 9: Details of the packages captured during the SSL traffic.

Table 1: Packet change between the client and the server.

Frame	Source Address	Destination Address	SSL Count	SSL Type
3900	172.28.173.205	3.221.181.255	1	Client Hello
4563	3.221.181.255	172.28.173.205	1	Server Hello
4565	3.221.181.255	172.28.173.205	1	Certificate
4566	3.221.181.255	172.28.173.205	2	Server Key Exchange, Server Hello Done
4572	172.28.173.205	3.221.181.255	3	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
4578	172.28.173.205	3.221.181.255	1	Application Data
4579	172.28.173.205	3.221.181.255	1	Application Data
4774	3.221.181.255	172.28.173.205	3	New Session Ticket, Change Cipher Spec, Encrypted Handshake Message

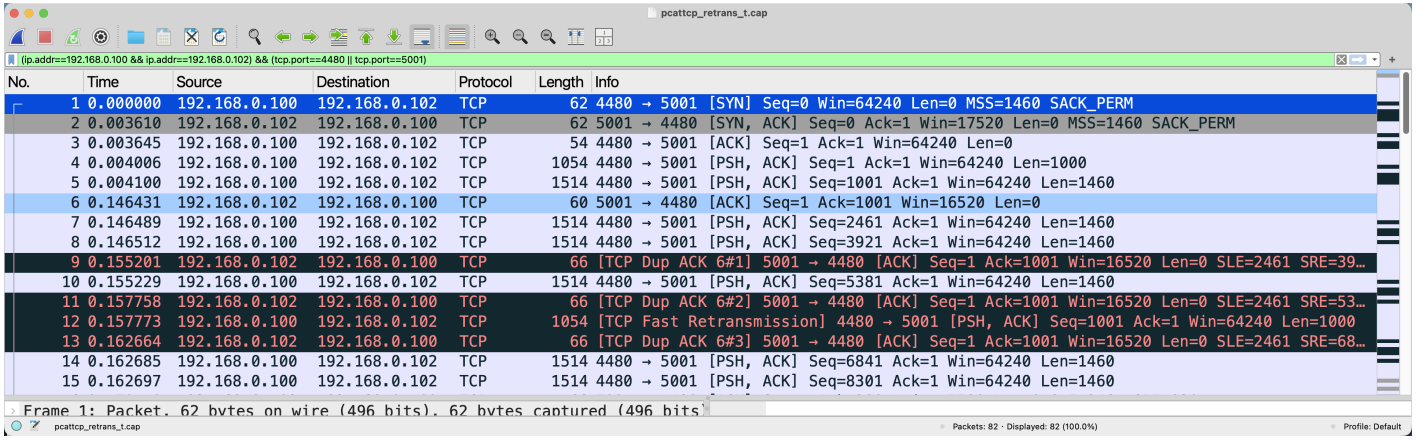


Figure 10: All packets in `pcattcp_retrans.t.cap`.

Problem 4: Analyzing TCP Traffic

The `pcattcp_retrans.t.cap` file is opened and the packets are shown in Figure 10.

Q1: The information of the handshake between `192.168.0.100:4480` and `192.168.0.102:5001` is shown in Table 2.

Q2: As shown in Figure 10, the length of the TCP segment (packet No. 4) is 1054 Bytes.

Q3: The RTT is the time between when the data segment is sent and the time when it is acknowledged. Hence, according to Figure 10, it is calculated as:

$$\text{RTT} = 0.146431 - 0.004006 = 0.142425 \text{ s.}$$

Table 2: The information of the handshake between `192.168.0.100:4480` and `192.168.0.102:5001`.

Frame	Source Address	Destination Address	Protocol	Length	Information
1	192.168.0.100	192.168.0.102	TCP	62 Bytes	[SYN]
2	192.168.0.102	192.168.0.100	TCP	62 Bytes	[SYN, ACK]
3	192.168.0.100	192.168.0.102	TCP	54 Bytes	[ACK]